マイコンシステム
第１１回

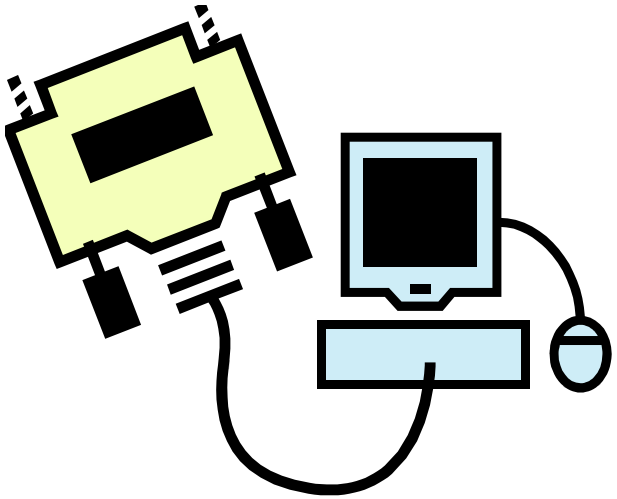
青森大学 ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

目次

- 講義
  - 内部設計②
- 例題
  - 定義した機能を実現する方法を検討する
- 課題
  - 時間を表示する機能を検討する



講義

組込みシステム開発４

内部設計②

組込みシステム開発

週	テーマ	内容
8	要求分析・外部設計	キッチンタイマーに必要な機能を抽出 画面表示、ボタンを押した時の動作を設定 機能の実現手順をフローチャートで図示
9		
10	内部設計	状態遷移図・状態遷移表の使い方 システムの内部構造を考える 機能の実現手順をより詳細に決める
11		
12		
13	実装	プログラムを作る
14		
15	テスト・完成	設計通りにできているか動作検証する

内部設計

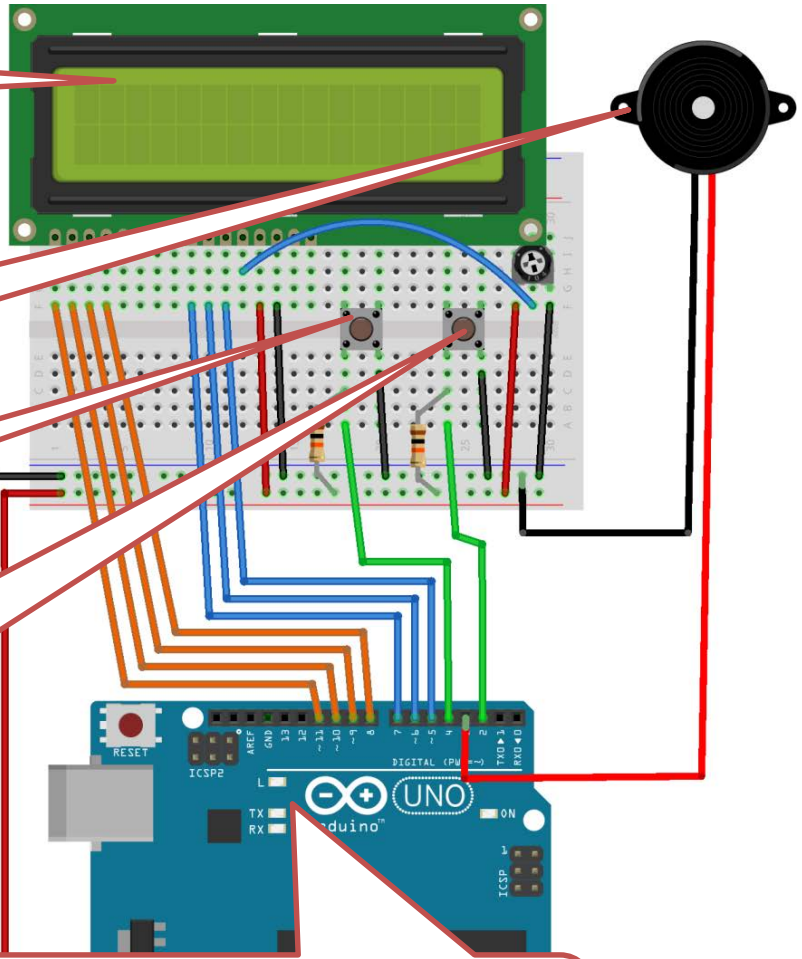
- 内部設計
  - 外部設計で定義した機能を、プログラムでどのように実現するか明確にする
  - 開発者の視点から、システム内部でどのような処理を行うかを考える
  - プログラムを共通化できそうな部分を検討

開発・実行環境

- 開発環境
  - Windows PC　(Windows 10 ～)
  - arduino IDE 1.8.9
  - 言語：arduino言語（C言語相当）

- 実行環境
  - マイコン arduino UNO Rev.3
  - 外部回路（スイッチ×2、ブザー、LCD）

ハードウェアの確認



液晶ディスプレイ
時間と状態の表示

ブザー
アラームを鳴らす・止める

スイッチ2
分ボタンで計測時間を設定

スイッチ1
スタート・ストップスイッチで
カウントダウン開始・一時停止

LED（マイコン上）
アラームを鳴らす・止める

例題

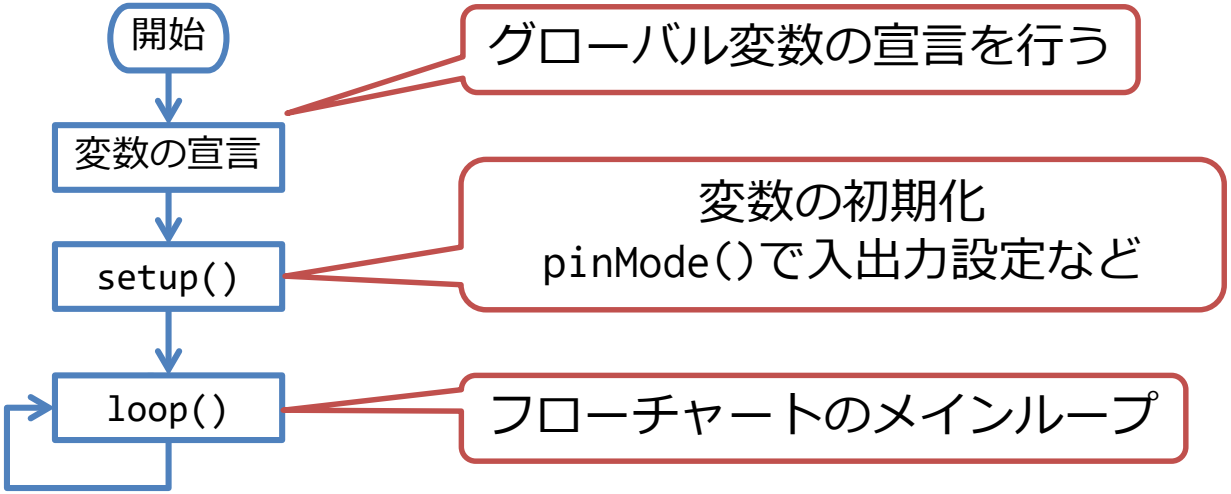
定義した機能を

C言語で実現する

方法を検討する

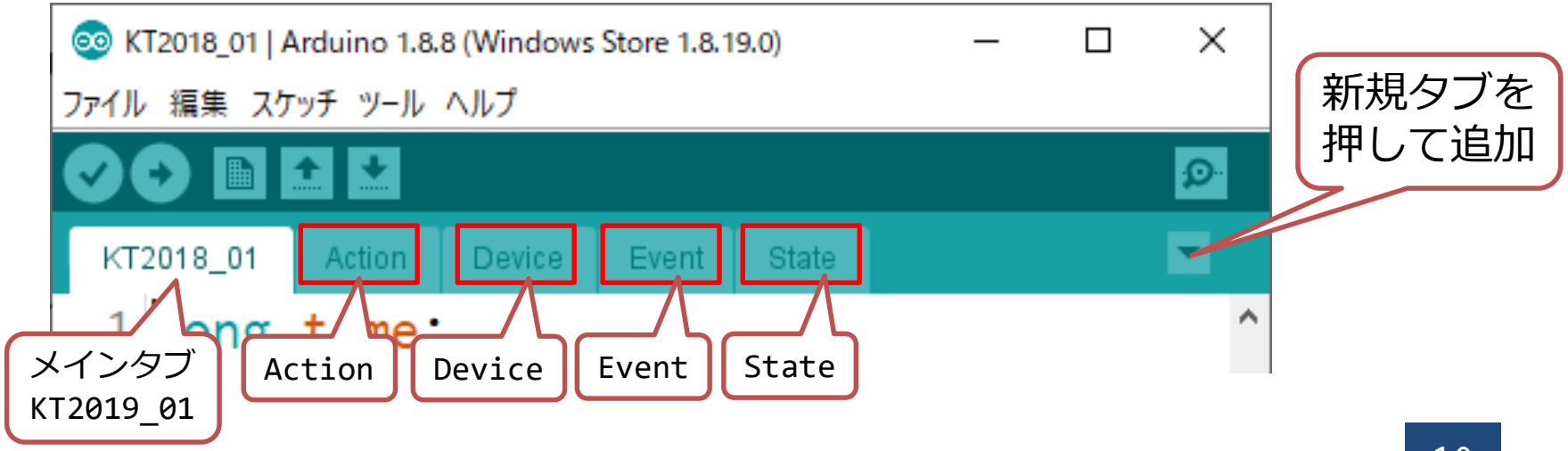
メインタブ

- 前回作成のフローチャートをベースに実装
- メインタブ冒頭で、グローバル変数の宣言
- setup()でグローバル変数の初期化、デバイスの初期化
- loop()で繰り返しを行う



ファイル分割

- プログラムを複数のタブに分けて実装する
  - メインタブ(変数の宣言、setup()関数，loop()関数の定義)
  - Action(各アクションの処理を行う関数を定義)
  - Device(スイッチ、ブザー、LCDの設定や動作を行う関数を定義)
  - Event(各イベント判定の関数を定義)
  - State(各状態の処理を関数で定義)



Actionタブ

- 各アクションをvoid型関数で定義
  - 設定時間を1分増加する　void addOneMinute(){}　１分増加
  - カウントダウンを開始する　void startTimer(){}　カウントダウンを開始する
  - カウントダウンを停止する　void stopTimer(){}　カウントダウンを停止する
  - アラームを鳴らす　void startAlarm(){}　アラームを鳴らす
  - アラームを止める　void stopAlarm(){}　アラームを止める
  - 時間を表示する(追加)　void displayTimer(long ms){}

```
//Actionタブ
void addOneMinute(){}                    //設定時間を１分増加する
void startTimer(){}                      //カウントダウンを開始する
void stopTimer(){}                       //カウントダウンを停止する
void startAlarm(){}                      //アラームを鳴らす
void stopAlarm(){}                       //アラームを止める
void displayTimer(long ms){}             //時間を表示する
```

Deviceタブ

- 第7回課題を参考に以下の関数を定義

```
//Deviceタブ
#include <LiquidCrystal.h>
LiquidCrystal lcd(5, 6, 7, 8, 9, 10, 11);
const int SW1Pin = 2;
const int SW2Pin = 4;
const int LEDPin = 13;
const int SPKPin = 3;
void setupDevice(){}                                    //初期設定用関数(追加)
void displayString(String str1, String str2){}          //LCDに文字列を表示
boolean SW1IsPressed(){}                                //SW1を押したか
boolean SW2IsPressed(){}                                //SW2を押したか
void startBuzzer(){}                                    //ブザーを鳴らす
void stopBuzzer(){}                                     //ブザーを止める
void lightOn(){}                                        //LEDを点灯する
void lightOff(){}                                       //LEDを消灯する
```

Eventタブ

- 各イベント判定をboolean型関数で定義
  - 分ボタンを押したか


```
boolean MinuteButtonIsPressed(){}
```

  - StartStopボタンを押したか


```
boolean StartStopButtonIsPressed(){}
```

  - 計測時間が0になったか


```
boolean TimeIsUp(){}
```

```
//Eventタブ
boolean MinuteButtonIsPressed(){}          //分ボタンを押したか
boolean StartStopButtonIsPressed(){}       //StartStopボタンを押したか
boolean TimeIsUp(){}                       //計測時間が0になったか
```

Stateタブ・メインタブ

- Stateタブは空欄
  (次回以降追加)

- メインタブ
  - ファイル名 KT2018_01
  - setup()関数
  - loop()関数

```
//Stateタブ
```

```
//メインタブ KT2018_01


void setup(){}
void loop(){}
```

ここまで作業した時点で
検証ボタンを押してエラーがないか確認

例題のまとめ

```
//メインタブ KT2019_01
void setup(){}
void loop(){}
```

```
//Actionタブ
void addOneMinute(){}
void startTimer(){}
void stopTimer(){}
void startAlarm(){}
void stopAlarm(){}
void displayTimer(long ms){}
```

```
//Eventタブ
boolean MinuteButtonIsPressed(){}
boolean StartStopButtonIsPressed(){}
boolean TimeIsUp(){}
```

```
//Stateタブ
```

```
//Deviceタブ
#include <LiquidCrystal.h>
LiquidCrystal lcd(5,6,7,8,9,10,11);
const int SW1Pin = 2;
const int SW2Pin = 4;
const int LEDPin = 13;
const int SPKPin = 3;

void setupDevice(){}
void displayString(String str1, String
str2){}
boolean SW1IsPressed(){}
boolean SW2IsPressed(){}
void startBuzzer(){}
void stopBuzzer(){}
void lightOn(){}
void lightOff(){}
```

課題

時間を表示する機能を検討する

動作確認

課題　時間を表示する機能を検討する

- 計測時間はミリ秒で管理
  - グローバル変数として定義
  - 変数名 `time`, 型は長整数型(`long`)
    - 初期化する場合、整数の後ろにL(エル)をつける
    - 例 0で初期化する場合 `time = 0L;`
- 時間を表示する `void displayTimer(long ms)`
  - 分2桁、秒2桁、ミリ秒3桁で表示する
    - 表示例　`00:00.000`
  - パラメータ `long ms` 表示する時間(ミリ秒)

どうやって時間を表示するか

① 時間(ミリ秒)から、分・秒・ミリ秒に変換

- 分は時間を1000で割り、さらに60で割った答
- 秒は時間を1000で割り、60で割った余り
- ミリ秒は時間を1000で割った余り
- 除算(/)・剰余(%)の計算を使う
- 長整数型から整数型へキャスト(型変換)する

② 分:秒.ミリ秒を連結して文字列にする

- 分が1桁の場合、分の10の位に0を追加
- 秒が1桁の場合、秒の10の位に0を追加
- ミリ秒が2桁の場合、ミリ秒の100の位に0を追加
- ミリ秒が1桁の場合、ミリ秒の100の位と10の位に00を追加

③ DeviceタブのdisplayString()関数を使って表示する

確認用プログラム　（一部空欄）

追加

```
//メインタブ KT2018_01
long time;                  //計測時間timeの宣言

void setup() {
  setupDevice();            //入出力設定、初期設定実行
  time = 0L;                //時間を0に初期化する
}
void loop() {
  displayTimer(time);       //時間を表示する
  delay(500);               //500ミリ秒待機
}
```

追加

追加

```
//Eventタブ　（そのまま）
boolean MinuteButtonIsPressed(){}
boolean StartStopButtonIsPressed(){}
boolean TimeIsUp(){}
```

```
//Stateタブ（そのまま）
```

```
//Deviceタブ
#include <LiquidCrystal.h>
LiquidCrystal lcd(5, 6, 7, 8, 9, 10, 11);
const int SW1Pin = 2;
const int SW2Pin = 4;
const int LEDPin = 13;
const int SPKPin = 3;
void setupDevice(){           //初期設定用関数
  pinMode(SW1Pin, INPUT);     //SW1Pinを入力設定
  pinMode(SW2Pin, INPUT);     //SW2Pinを入力設定
  pinMode(LEDPin, OUTPUT);    //LEDPinを出力設定
  lcd.begin(16, 2);           //LCDの設定
  lcd.clear();                //画面消去
  Serial.begin(9600);         //シリアルモニタ開始
}
void displayString(String str1, String str2){
  lcd.clear();            //画面消去
  lcd.setCursor(0, 0);    //上段左端に移動
  lcd.print(str1);        //文字列str1を表示
  lcd.setCursor(0, 1);    //下段左端に移動
  lcd.print(str2);        //文字列str2を表示
}
以下省略（ほかの関数はそのまま）
```

追加

追加

確認用プログラム　（一部空欄）

```
//Actionタブ
void addOneMinute(){}     //設定時間を１分増加する
void startTimer(){}       //カウントダウンを開始する
void stopTimer(){}        //カウントダウンを停止する
void startAlarm(){}       //アラームを鳴らす
void stopAlarm(){}        //アラームを止める

void displayTimer(long ms){            //時間を表示する
  int minute = int((ms/1000L)/60L);    //分を求める
  int second = int((ms/1000L)%60L);    //秒を求める
  int msec   = int(ms % 1000L);        //ミリ秒を求める

  String str1 = "";

  //分が1桁の場合
  if(minute [    ] && minute [    ] ){
    //10の位に0を追加
    str1 += String("0");
  }
  //分を文字列str1に追加する
  str1 += String(minute);
  //分と秒の間に:を入れる
  str1 += String(":");
```

追加

```
  //秒が1桁の場合
  if(second [    ] && second [    ]){
    //10の位に0を追加
    str1 += String("0");
  }
  //秒を文字列str1に追加する
  str1 += String(second);
  //秒とミリ秒の間に .を追加
  str1 += String(".");
  //ミリ秒が2桁以下の場合
  if(msec [    ]  && msec [    ] ){
    //100の位に0を追加
    str1 += String("0");
  }
  //ミリ秒が1桁以下の場合
  if(msec[    ]  && msec [    ]){
    //10の位に0を追加
    str1 += String("0");
  }
  //ミリ秒を文字列str1に追加する
  str1 += String(msec);
  //時間を1行目に表示、空白2行目に表示
  displayString(str1, "");
}
```

追加

条件の空欄に何を入れるか

- 変数(minute,second,msec)が1桁の場合

  ⇒ 0,1,2,3,4,5,6,7,8,9の場合

  ⇒ 0以上10より小さい場合

- 変数msecが2桁以下の場合

  ⇒ 0,1,2,・・・97,98,99の場合

  ⇒ 0以上100より小さい場合

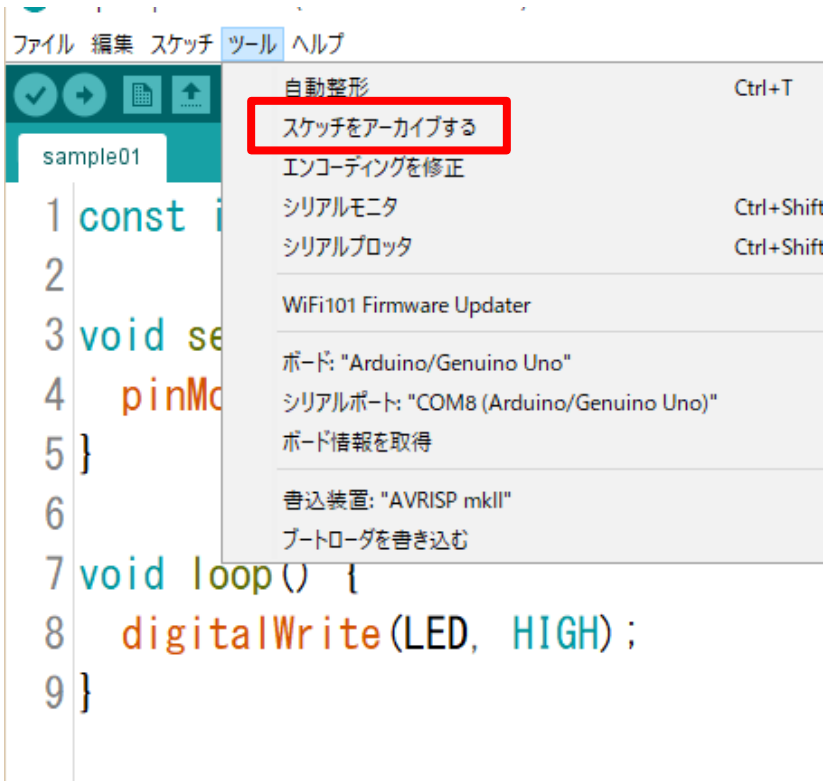
動作確認

- 分2桁、秒2桁、ミリ秒3桁で表示できるか
- timeの初期化する値を変更して、以下の複数パターンで正しく表示できるか確認する
  - time = 0L 00:00.000
  - time = 1L 00:00.001 (ミリ秒1桁)
  - time = 12L 00:00.012 (ミリ秒2桁)
  - time = 123L 00:00.123 (ミリ秒3桁)
  - time = 1234L 00:01.234 (秒1桁)
  - time = 12345L 00:12.345 (秒2桁)
  - time = 61234L 01:01.234 (分1桁)
  - time = 661234L 11:01.234 (分2桁)

課題の提出方法

プログラムをアーカイブする

- ツール ->
  スケッチをアーカイブ



- 圧縮ファイル名を指定

課題 提出先・期限

- 圧縮ファイルをメールに添付して提出する
  - 課題プログラム KT2018_01
  - 提出前に動作確認してから提出すること
- 提出先
  - haship@aomori-u.ac.jp
  - 件名：マイコンシステム11週
  - 本文に学籍番号、氏名を書く
- 提出期限
  - 次回授業前まで